

Dynamic Programming

Coin Change

- Given an integer array of coins[] of size N representing different types of currency and an integer sum, The task is to find the number of ways to make the sum by using different combinations from coins[].
- Note: Assume that you have an infinite supply of each type of coin, and you are allowed to use same valued multiple times.

Coin Change

- ***Input:*** $sum = 4$, $coins[] = \{1, 2, 3\}$, $n = 3$

Output: 4

Explanation: there are four solutions:

- $\{1, 1, 1, 1\}$, $\{1, 1, 2\}$, $\{2, 2\}$, $\{1, 3\}$.

- ***Input:*** $sum = 10$, $coins[] = \{2, 5, 3, 6\}$, $n = 4$

Output: 5

Explanation: There are five solutions:

- $\{2, 2, 2, 2, 2\}$, $\{2, 2, 3, 3\}$, $\{2, 2, 6\}$, $\{2, 3, 5\}$ and $\{5, 5\}$.

Coin Change

For every single coin, you have two choices:

Exclude the coin: Try to make the target amount using only the remaining types of coins.

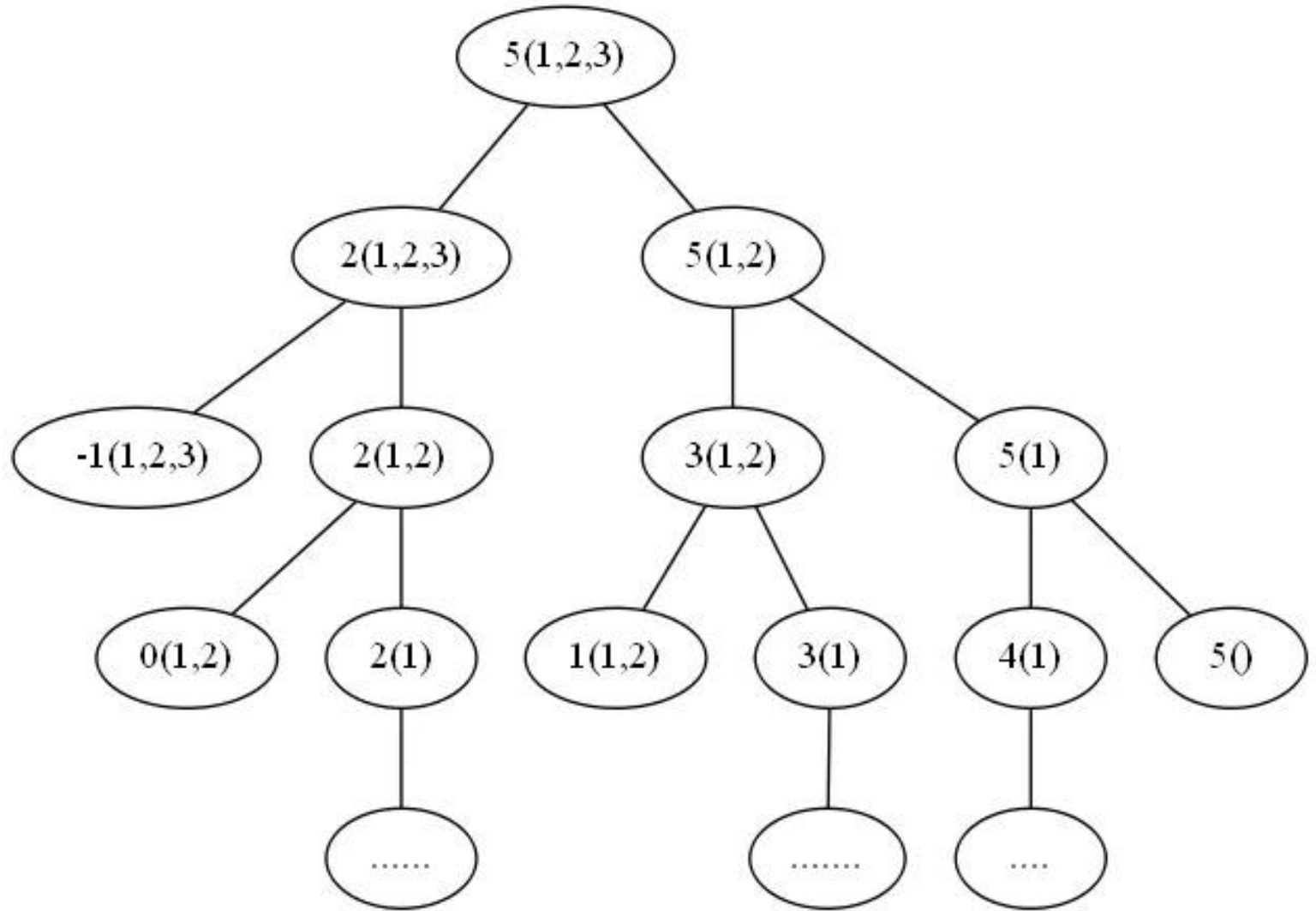
Include the coin: Use the coin once (subtract its value from the target) and keep the option open to use it again.

Coin Change

- Recurrence Relation:

- $\text{change}(\text{sum}, n) = \text{change}(\text{sum} - \text{coins}[n], n) + \text{change}(\text{sum}, n-1)$

Coin Change



Coin Change Bottom-Up (Tabulation)

	Sum = 0	Sum = 1	Sum = 2	...	...		Sum = S
0 coin							
1 coins							
2 coins							
...							
...							
n coins							

Coin Change Bottom-Up (Tabulation)

	Sum = 0	Sum = 1	Sum = 2	...	...		Sum = S
0 coin	1	0	0	0	0	0	0
1 coins	1						
2 coins	1						
...	1						
...	1						
	1						
	1						
n coins	1						

Bottom up implementation 1

```
for (i = 0; i <= n; i++)
    dp[i][0] = 1;
for (j = 1; j <= sum; j++)
    dp[0][j] = 0;
for (i = 1; i <= n ; i++) {
    for (j = 1; j <= sum; j++) {
        if(j < coins[i - 1]) //0 indexed coins array
            dp[i][j] = dp[i-1][j];
        else
            dp[i][j] = dp[i-1][j] + dp[i] [ j - coins[i - 1] ];
    }
}
return dp[n][sum];
```

Coin Change Bottom-Up (2)

	Sum = 0	Sum = 1	Sum = 2	...	...			Sum = S
0 coin	1	0	0	0	0	0	0	0
1 coins	0	0	0	0	0	0	0	0
2 coins	0	0	0	0	0	0	0	0
...	0	0	0	0	0	0	0	0
...	0	0	0	0	0	0	0	0
	0	0	0	0	0	0	0	0
	0	0	0	0	0	0	0	0
n coins	0	0	0	0	0	0	0	0

Bottom up implementation (2)

```
dp[0][0] = 1;
```

```
for (i = 1; i <= n ; i++) {  
    for (j = 0; j <= sum; j++) {  
        if(j < coins[i - 1]) //0 indexed coins array  
            dp[i][j] = dp[i-1][j];  
        else  
            dp[i][j] = dp[i-1][j] + dp[i] [ j - coins[i - 1] ];  
    }  
}  
return dp[n][sum];
```

Top down solution?

Top down solution (Recursive)

- Initialize the dp array as -1 //to check whether the value is updated or not
- f(n, sum)
- Base Cases:
 - if(sum == 0) return dp[sum][n] = 1;
 - else if(n == 0) return dp[sum][n] = 0;
 - else if(dp[n][sum] != -1) return dp[n][sum];
 - else recursive call

Coin Change (Variations)

- Can only use each coins only once
- Each coins has limited supply (Can be different for each coin type)
- Only allowed to use k coins (all together)

Coin Change (Minimum Number of coins)

- Your goal is to find the minimum number of coins required, to give an exchange of a target (sum) by using n coins having values:
 - $\text{coins[]} = \{c_1, c_2, c_3, \dots, c_n\}$

Recurrence Relation

- Smaller sub problems and the relation between them ?

Recurrence Relation

< Excluding n^{th} coins and including n^{th} coins >

- Excluding:
 - $dp[n-1][sum]$
- Including:
 - $1 + dp[n][sum - coins[n]]$

Relation between them?

Coin Change (Minimum Number of coins)

< Excluding n^{th} coins and including n^{th} coins >

- Excluding:
 - $dp[n-1][sum]$
- Including:
 - $1 + dp[n][sum - coins[n]]$

$\min(\text{excluding}, \text{including})$

Coin Change (Minimum number of coins)

	Sum = 0	Sum = 1	Sum = 2	...	...		Sum = S
0 coin							
1 coins							
2 coins							
...							
...							
n coins							

Coin Change (Minimum number of coins)

	Sum = 0	Sum = 1	Sum = 2	...	...		Sum = S
0 coin	0						
1 coins	0						
2 coins	0						
...	0						
...	0						
	0						
	0						
n coins	0						

Coin Change (Minimum number of coins)

	Sum = 0	Sum = 1	Sum = 2	...	...		Sum = S
0 coin	0	∞	∞	∞	∞	∞	∞
1 coins	0						
2 coins	0						
...	0						
...	0						
	0						
	0						
n coins	0						

Bottom up implementation 1

```
for (i = 0; i <= n ; i++) {  
    for (j = 0; j <= sum; j++) {  
        if(j==0) dp[i][j] = 0; //Can be initialized outside loop  
        else if(i==0) dp[i][j] = INF; //Can be initialized outside loop  
  
        else if(j < coins[i - 1])  
            dp[i][j] = dp[i-1][j];  
        else  
            dp[i][j] = min( dp[i-1][j], 1 +dp[i] [ j - coins[i - 1] ] );  
    }  
}
```

Knapsack Problem [0-1]

- Given weights **and** values of N items, put these items in a knapsack of capacity W to get the maximum total value in the knapsack.

Knapsack Problem [0-1]

- Input: $N = 3$, $W = 4$
- $\text{values[]} = \{1, 2, 3\}$
- $\text{weight[]} = \{4, 5, 1\}$
- Output: 3

- Input: $N = 3$, $W = 3$
- $\text{values[]} = \{1, 2, 3\}$
- $\text{weight[]} = \{4, 5, 6\}$
- Output: 0

Recurrence Relation

- Smaller sub problems and the relation between them ?

Recurrence Relation

- Smaller sub problems and the relation between them ?
 - $dp[i][w] = \max(val[i] + dp[i - 1][w - wt[i]], dp[i - 1][w]);$

Knapsack Problem [0-1]

	$W = 0$	$W = 1$	$W = 2$	...	...		$W = S$
0 obj							
1 obj							
2 obj							
...							
...							
n obj							

Knapsack Problem [0-1]

	$w = 0$	$w = 1$	$w = 2$	...	...		$w = W$
0 obj	0	0	0	0	0	0	0
1 obj	0						
2 obj	0						
...	0						
...	0						
	0						
	0						
n obj	0						

0/1 Knapsack Problem

- Capacity = 8 kg
- $N = 4$
- Weight of each object: {3, 4, 6, 5}
- Value of each object: {2, 3, 1, 4}

0/1 Knapsack Problem

Weight	Value		0	1	2	3	4	5	6	7	8
		0	0	0	0	0	0	0	0	0	0
3	2	1	0	0	0	2	2	2	2	2	2
4	3	2	0	0	0	2	3	3	3	5	5
6	1	3	0	0	0	2	3	3	3	5	5
5	4	4	0	0	0	2	3	4	4	5	6

Knapsack Problem Variations

- Each object has unlimited supply
- You may take a fraction of a object